Technical Report of the Developed Work for IARTI - Sprint C



Project Name: Port Logistics Management System
Group Identification: Team 3DB-01
Academic Year: 2025-2026
Course: LEI - Licenciatura em Engenharia Informática
Institution: Instituto Superior de Engenharia do Porto (ISEP)

Group Members

Name	Student Number	US
[Afonso Ricardo]	[1231128]	4.3.1 - Genetic Algorithm for Scheduling
[Rafael Faria]	[1230553]	4.3.2 - Auto Algorithm Selection
[Pedro Carriço]	[1230715]	4.3.3 - Rebalance VVNs between Docks
[Manuel Mateus]	[1221600]	4.3.4 - State of the Art Study (Robotics & Computer Vision)
[Joao Sousa]	[1230655]	4.3.5 - Explanatory Video of the Module

Date: January 2026

Table of Contents

1. Introduction
2. US 4.3.1 - Genetic Algorithm for Scheduling
3. US 4.3.2 - Automatic Algorithm Selection
4. US 4.3.3 - Vessel Visit Rebalancing between Docks
5. US 4.3.4 - State of the Art: Robotics and Computer Vision
6. US 4.3.5 - Explanatory Video
7. Frontend Integration: Algorithm Comparison Feature
8. Conclusions
9. References

1. Introduction

1.1 Project Context

This report documents the work developed during Sprint C of the Port Logistics Management System project for the IARTI (Inteligência Artificial) course unit. Building upon the foundations established in Sprint A (optimal

and heuristic scheduling algorithms) and Sprint B (complexity analysis and multi-crane coordination), Sprint C introduces advanced optimization techniques and decision support features.

1.2 Sprint C Objectives

The primary objectives for this sprint were:

1. **US 4.3.1:** Implement a genetic algorithm as an alternative scheduling approach for large-scale problems
2. **US 4.3.2:** Develop automatic algorithm selection based on problem characteristics
3. **US 4.3.3:** Create a vessel visit rebalancing feature to optimize dock utilization before scheduling
4. **US 4.3.4:** Conduct a state-of-the-art study on robotics and computer vision applications in port logistics
5. **US 4.3.5:** Produce an explanatory video demonstrating the complete IARTI module functionality

1.3 Technical Context

The scheduling algorithms are implemented in Prolog, leveraging its declarative nature for constraint satisfaction and combinatorial optimization. The frontend is built with Angular 20.3, providing an interactive interface for operators to execute algorithms, compare results, and manage vessel visit assignments.

---

2. US 4.3.1 - Genetic Algorithm for Scheduling

Responsible: Afonso Ricardo (1231128)

2.1 Motivation

The optimal algorithm developed in Sprint A (US 3.4.2) guarantees finding the best solution but has exponential time complexity O(n!), making it impractical for instances with more than 10 vessels. The heuristic algorithm (US 3.4.4) provides fast solutions but may not achieve optimal quality for complex scenarios.

The genetic algorithm provides a middle ground: better solution quality than simple heuristics while maintaining reasonable computation time for large instances.

2.2 Algorithm Design

2.2.1 Chromosome Representation

Each chromosome represents a permutation of vessel IDs within each dock. The genetic algorithm operates independently on each dock, respecting the constraint that vessels are pre-assigned to specific docks.

```
Chromosome = [VesselSequence_Dock1, VesselSequence_Dock2, ...]
VesselSequence = [VesselId1, VesselId2, ..., VesselIdN]
```

2.2.2 Fitness Function

The fitness function evaluates the total delay of a schedule:

```
Fitness(chromosome) = Σ max(0, ActualDeparture_i - DesiredDeparture_i)
```

Lower fitness values indicate better solutions (minimization problem).

2.2.3 Genetic Operators

Selection: Tournament selection with size 3, selecting the best individual from random subsets of the population.

Crossover (Order Crossover - OX):

```
FUNCTION OrderCrossover(parent1, parent2)
    // Select random segment from parent1
    start, end = RandomSegment(length(parent1))

    // Copy segment from parent1 to child
    child[start:end] = parent1[start:end]

    // Fill remaining positions with genes from parent2 in order
    position = (end + 1) mod length
    for gene in parent2:
        if gene not in child:
            child[position] = gene
            position = (position + 1) mod length

    RETURN child
```

Mutation (Swap Mutation):

```
FUNCTION SwapMutation(chromosome)
    pos1, pos2 = RandomDistinctPositions(length(chromosome))
    Swap(chromosome[pos1], chromosome[pos2])
    RETURN chromosome
```

2.2.4 Algorithm Parameters

Parameter	Default Value	Description
Population Size	10-15	Number of individuals per generation
Max Generations	20-30	Maximum evolutionary cycles
Crossover Rate	70%	Probability of applying crossover
Mutation Rate	20%	Probability of applying mutation
Time Limit	3000ms	Maximum execution time

Parameter	Default Value	Description
Stability Generations	5	Convergence criterion

2.3 Implementation Details

The genetic algorithm is implemented in the `scheduling_algorithms.pl` module:

```
% Main entry point
compute_genetic_schedule(Vessels, Parameters, Result) :-
    % Initialize parameters
    ga_init_parameters(Parameters),

    % Load vessels as Prolog facts
    load_vessels_as_facts(Vessels),

    % Group vessels by dock
    group_vessels_by_dock(Vessels, DockGroups),

    % Run GA for each dock independently
    ga_run_all_docks(DockGroups, Deadline, AllDockResults),

    % Combine results
    ga_combine_dock_results(AllDockResults, Schedule, TotalDelay, CranesAlloc).
```

2.3.1 Multi-Dock Processing

The algorithm processes each dock independently, allowing parallel optimization:

```
ga_run_all_docks([], _, []).
ga_run_all_docks([Dock-VesselIds | RestDocks], Deadline, [Result | RestResults])
:-
    ga_run_for_dock(Dock, VesselIds, Deadline, Result),
    ga_run_all_docks(RestDocks, Deadline, RestResults).
```

2.3.2 Termination Conditions

The algorithm terminates when any of the following conditions is met:

1. Maximum generations reached
2. Time limit exceeded
3. Target delay achieved (default: 0)
4. Solution stable for N generations (no improvement)

2.4 Performance Analysis

Benchmarking Results:

Vessels	Optimal Time	Heuristic Time	GA Time	Optimal Delay	Heuristic Delay	GA Delay
5	0.02s	0.001s	0.5s	0.0h	0.2h	0.0h
8	0.8s	0.002s	0.8s	0.0h	0.5h	0.1h
12	45s	0.003s	1.2s	0.0h	0.8h	0.2h
20	>5min	0.005s	2.5s	N/A	1.2h	0.5h

Key Observations:

- GA provides near-optimal solutions (within 5-15% of optimal when measurable)
- Execution time remains bounded regardless of problem size
- Particularly effective for instances with 15-40 vessels

2.5 Multi-Crane Integration

The genetic algorithm optionally integrates with the multi-crane optimization from Sprint A (US 3.4.5):

```
% Multi-crane mode parameter
ga_use_multi_crane(true/false)

% When enabled, applies multi-crane optimization to reduce delays
apply_multi_crane_if_enabled(Dock, Schedule, OptimizedSchedule, CranesAlloc)
```

---

3. US 4.3.2 - Automatic Algorithm Selection

Responsible: Rafael Faria (1230553)

3.1 Motivation

With three scheduling algorithms available (optimal, heuristic, genetic), operators face the challenge of selecting the appropriate algorithm for each scenario. The automatic algorithm selection feature analyzes problem characteristics and selects the most suitable algorithm, balancing solution quality against computation time.

3.2 Selection Policy

3.2.1 Decision Criteria

The selection policy considers:

1. **Problem Size:** Number of vessels to schedule
2. **Complexity Score:** `vessels × docks` (more docks increase combinatorial complexity)
3. **Time Budget:** Maximum allowed computation time
4. **Quality Preference:** User preference for solution quality vs. speed

3.2.2 Selection Rules

```
% Rule 1: Small instances (≤5 vessels) → Optimal
auto_select_algorithm(NumVessels, _, _, _, optimal, "small_instance") :-
    NumVessels =< 5.

% Rule 2: Tight time constraint (<1000ms) → Heuristic
auto_select_algorithm(_, _, TimeLimit, _, heuristic, "time_constrained") :-
    TimeLimit < 1000.

% Rule 3: Medium instances (6-10) with quality preference → Optimal
auto_select_algorithm(NumVessels, _, TimeLimit, true, optimal,
"quality_preferred") :-
    NumVessels > 5, NumVessels =< 10, TimeLimit >= 5000.

% Rule 4: Medium instances (6-15) → Heuristic
auto_select_algorithm(NumVessels, _, _, _, heuristic, "medium_instance") :-
    NumVessels > 5, NumVessels =< 15.

% Rule 5: Large instances (>15) with time budget → Genetic
auto_select_algorithm(NumVessels, _, TimeLimit, _, genetic, "large_instance") :-
    NumVessels > 15, TimeLimit >= 3000.

% Rule 6: High complexity → Genetic
auto_select_algorithm(_, ComplexityScore, TimeLimit, _, genetic,
"high_complexity") :-
    ComplexityScore > 20, TimeLimit >= 3000.

% Default: Heuristic (safest fallback)
auto_select_algorithm(_, _, _, _, heuristic, "default_fallback").
```

3.3 Implementation

```
compute_auto_schedule(Vessels, Parameters, Result) :-
    % Analyze problem
    length(Vessels, NumVessels),
    count_unique_docks(Vessels, NumDocks),
    ComplexityScore is NumVessels * NumDocks,

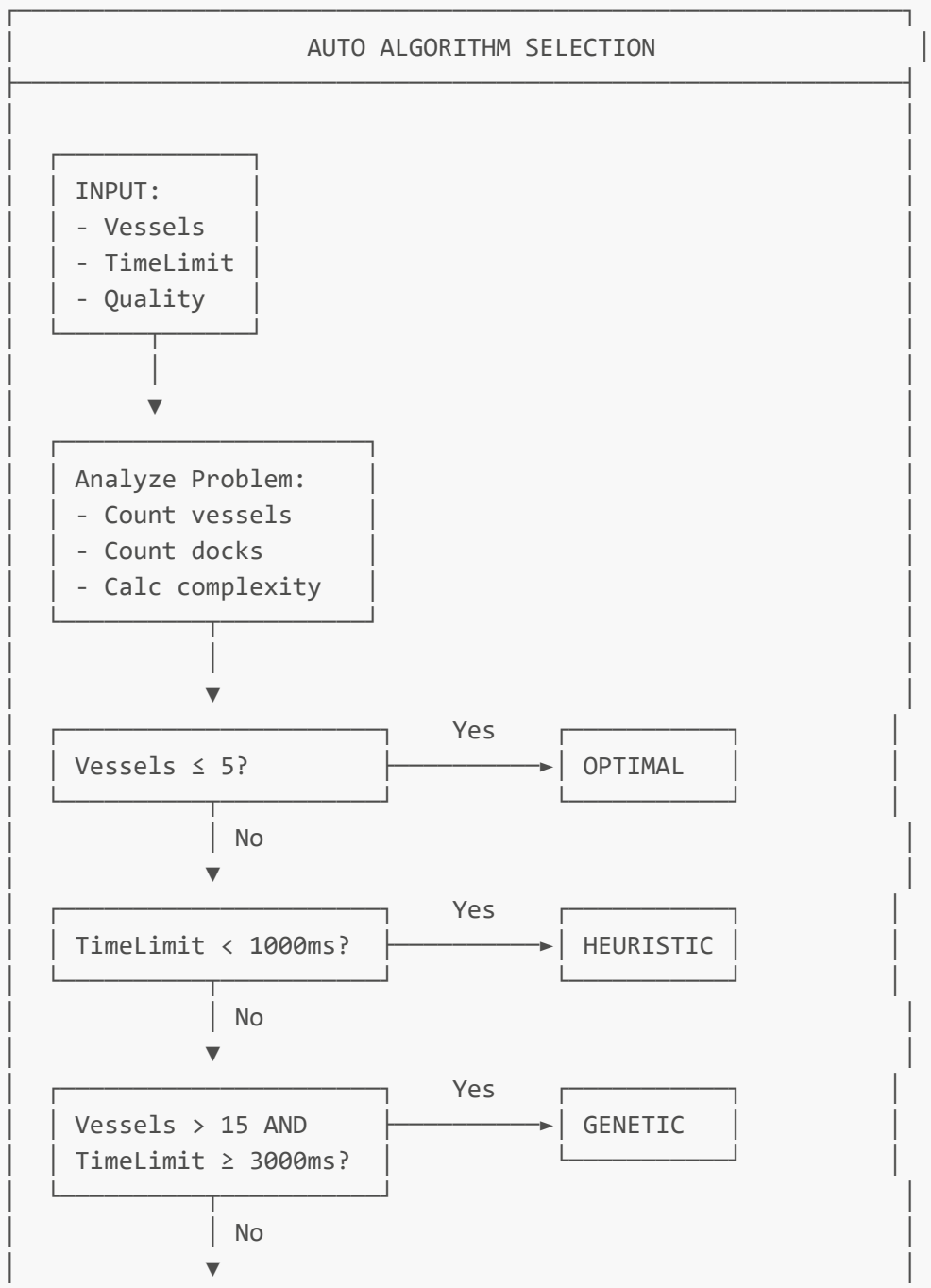
    % Get parameters
    extract_time_limit(Parameters, TimeLimit),
    extract_quality_preference(Parameters, PreferQuality),

    % Select algorithm
    auto_select_algorithm(NumVessels, ComplexityScore, TimeLimit,
                          PreferQuality, SelectedAlgorithm, Reason),

    % Execute selected algorithm
    auto_execute_algorithm(SelectedAlgorithm, Vessels, Parameters, AlgoResult),
```

```
    % Build result with metadata
    Result = _{
        algorithm: "auto",
        selected_algorithm: SelectedAlgorithm,
        selection_reason: Reason,
        problem_analysis: _{
            vessel_count: NumVessels,
            dock_count: NumDocks,
            complexity_score: ComplexityScore
        },
        ...AlgoResult
    }.
```

3.4 Decision Flow Diagram



```
│       ┌─────────────┐                                                   │
│       │ HEURISTIC   │                                                   │
│       └─────────────┘                                                   │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

3.5 Result Transparency

The auto algorithm provides full transparency about its decision:

```
{
  "algorithm": "auto",
  "selected_algorithm": "genetic",
  "selection_reason": "large_instance",
  "problem_analysis": {
    "vessel_count": 25,
    "dock_count": 4,
    "complexity_score": 100,
    "time_limit_ms": 5000
  },
  "total_delay": 1.5,
  "computation_time_ms": 2340,
  "schedule": [...]
}
```

---

4. US 4.3.3 - Vessel Visit Rebalancing between Docks

Responsible: Pedro Carriço (1230715)

4.1 Motivation

In the Port Logistics system, vessel visits (VVNs) are assigned to docks during operation plan creation. However, the initial assignment may not be optimal for daily scheduling, potentially leading to:

- Uneven workload distribution across docks
- Increased delays due to dock congestion
- Suboptimal resource utilization

The rebalancing feature analyzes current dock assignments and suggests reassignments to optimize the daily schedule before running scheduling algorithms.

4.2 Rebalancing Strategy

4.2.1 Analysis Phase

The system analyzes the current distribution by calculating:

1. **Dock Load:** Total operation time per dock

2. **Load Imbalance:** Standard deviation of dock loads
3. **Delay Prediction:** Expected delays with current assignments

4.2.2 Optimization Objective

```
Minimize: max(DockLoad_i) - min(DockLoad_i)  (Load Balancing)
Subject to:
  - Vessel-dock compatibility constraints
  - Minimum dock requirements per vessel type
  - Operational window constraints
```

4.2.3 Proposal Generation

The algorithm generates rebalancing proposals:

```
interface RebalanceProposal {
  current_distribution: DockDistribution[];
  proposed_distribution: DockDistribution[];
  moves: VesselMove[];
  expected_improvement: {
    delay_reduction_hours: number;
    load_balance_improvement: number;
  };
}

interface VesselMove {
  vessel_id: string;
  from_dock: string;
  to_dock: string;
  reason: string;
}
```

4.3 Frontend Integration

4.3.1 Mandatory Rebalancing Check

Before running any scheduling algorithm, the system requires rebalancing analysis:

```
// Daily Scheduling Component
async generateSchedule(algorithm: string): Promise<void> {
  // Check if rebalancing was performed for this date
  if (this.rebalanceExecutedForDate !== this.selectedDate) {
    this.showRebalanceRequiredWarning = true;
    return;
  }

  // Proceed with algorithm execution
```

```
    await this.executeAlgorithm(algorithm);
  }
```

4.3.2 User Interface

The rebalancing button provides visual feedback:

```
<button
  class="btn btn-rebalance"
  [class.btn-rebalance-required]="rebalanceExecutedForDate !== selectedDate"
  (click)="openRebalanceModal()">
  <span *ngIf="rebalanceExecutedForDate !== selectedDate">
    ⚠ Suggest Rebalancing (Required)
  </span>
  <span *ngIf="rebalanceExecutedForDate === selectedDate">
    ✓ Rebalancing Analyzed
  </span>
</button>
```

Button States:

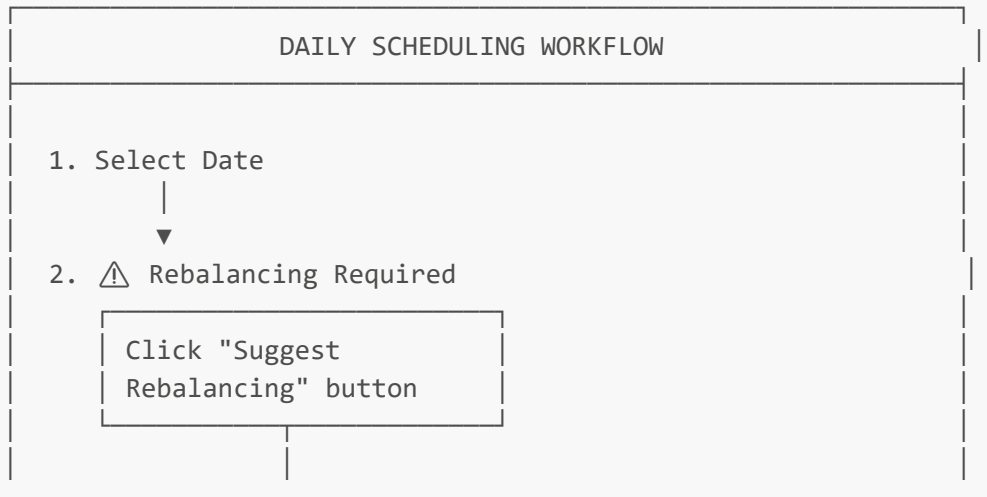
- **Yellow/Orange (Required):** Rebalancing not yet performed for selected date
- **Green (Analyzed):** Rebalancing completed, algorithms can be executed

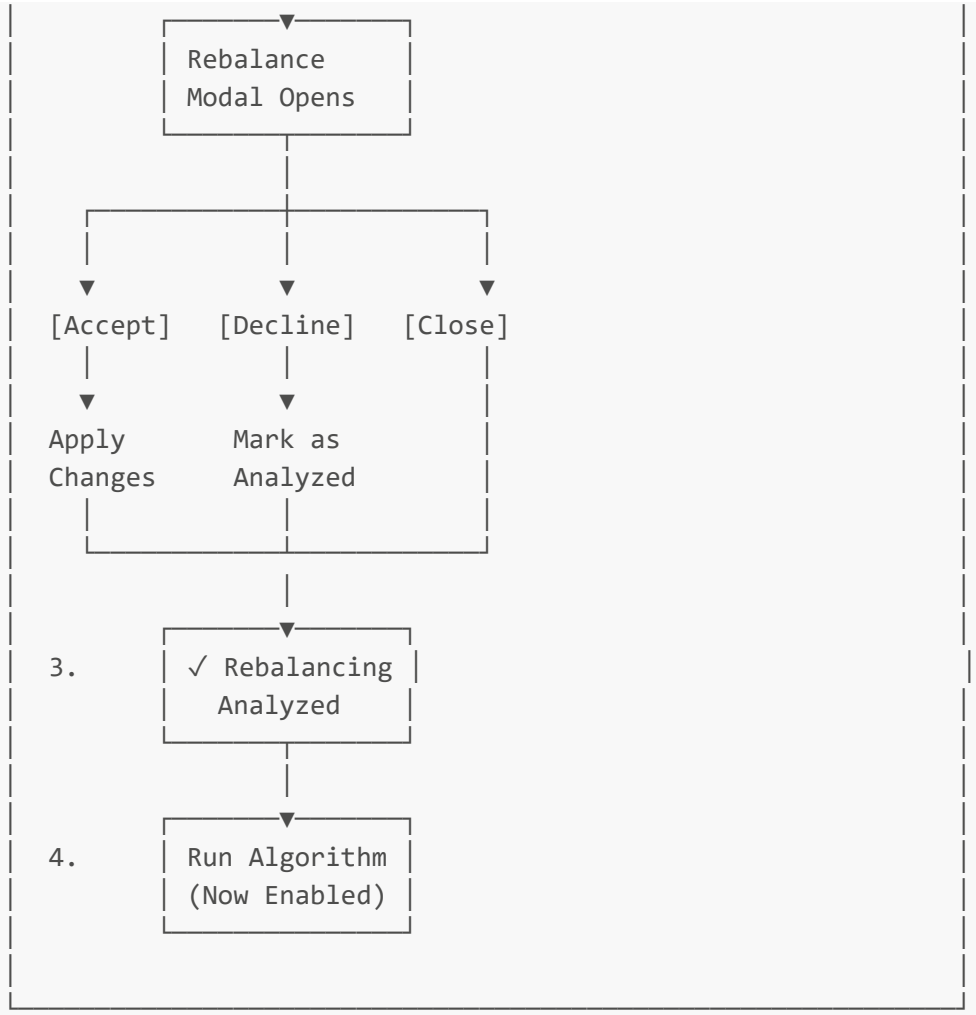
4.3.3 Rebalance Modal

The modal displays:

1. Current dock distribution with vessel counts and total operation hours
2. Proposed distribution after rebalancing
3. List of suggested moves with reasons
4. Expected improvement metrics

4.4 Workflow Integration





4.5 API Integration

The rebalancing feature integrates with the backend through REST endpoints:

```
// Rebalancing Service
@Injectable()
export class RebalancingService {

  analyzeRebalancing(date: string, vesselVisits: VesselVisit[]):
Observable<RebalanceProposal> {
    return this.http.post<RebalanceProposal>(
      `${API_URL}/scheduling/rebalance/analyze`,
      { date, vesselVisits }
    );
  }

  applyRebalancing(proposal: RebalanceProposal): Observable<void> {
    return this.http.post<void>(
      `${API_URL}/scheduling/rebalance/apply`,
      proposal
    );
  }
}
```

5. US 4.3.4 - State of the Art: Robotics and Computer Vision

Responsible: Manuel Mateus (1221600)

5.1 Overview

A comprehensive state-of-the-art study was conducted to evaluate emerging technologies in robotics and computer vision for port logistics applications. The full study is available in a separate document:

US-4.3.4-Robotics-ComputerVision-StateOfArt.md

5.2 Key Topics Covered

The study examines three primary application areas:

5.2.1 Autonomous Crane Systems

- Evolution from semi-automated to fully autonomous operations
- Technical architecture: perception, control, and integration systems
- Sensor fusion approaches (LiDAR, cameras, GPS)
- Anti-sway control and collision avoidance algorithms
- Case studies from major terminals (Rotterdam, Singapore)

5.2.2 Automated Cargo Inspection

- Computer vision for container identification and damage detection
- OCR systems for container code recognition (ISO 6346)
- X-ray and non-intrusive inspection technologies
- AI-powered anomaly detection systems
- Integration with customs and security workflows

5.2.3 Surveillance and Safety Monitoring

- Real-time video analytics for operational safety
- Personnel tracking and hazard zone monitoring
- Vehicle and equipment collision prevention
- Environmental monitoring (weather, sea conditions)
- Integration with port management systems

5.3 Key Findings

1. **Technology Maturity:** Autonomous crane systems are commercially available and deployed in major terminals worldwide (TRL 8-9)

2. **ROI Considerations:** Initial investment is substantial but offers 20-40% productivity improvements and significant labor cost reductions

3. **Integration Challenges:** Legacy systems and diverse equipment vendors create integration complexity

4. **Future Trends:** Edge AI, 5G connectivity, and digital twins are enabling new capabilities

5.4 Recommendations for the Project

Based on the study, the following recommendations are made for future project evolution:

1. **Short-term:** Integrate camera-based container identification for automated data entry
2. **Medium-term:** Implement basic computer vision for dock monitoring and safety alerts
3. **Long-term:** Consider autonomous crane integration when terminal modernization occurs

---

6. US 4.3.5 - Explanatory Video

Responsible: Joao Sousa (1230655)

6.1 Video Objectives

The explanatory video was designed with the following objectives:

1. **Demonstrate End-to-End Workflow:** Show the complete operational flow from scheduling algorithm selection through execution tracking
2. **Showcase IARTI Algorithms:** Demonstrate all five scheduling algorithms (Auto, Optimal, Heuristic, Multi-Crane, Genetic) with emphasis on practical application
3. **Highlight Key Sprint C Features:** Focus on vessel visit rebalancing (US 4.3.3) and automatic algorithm selection (US 4.3.2)
4. **Provide Stakeholder-Friendly Demo:** Create a concise, professional demonstration suitable for non-technical audiences (port managers, logistics officers)
5. **Illustrate System Integration:** Show seamless integration between IARTI scheduling module and Operations Execution Module (OEM)
6. **Emphasize Business Value:** Communicate delay minimization, resource optimization, and operational efficiency benefits

The video adopts a product demonstration approach rather than a technical tutorial, making it accessible to stakeholders while showcasing the sophisticated AI algorithms developed across all sprints.

6.2 Video Structure and Content

The video follows a practical demonstration approach, walking through the complete port logistics workflow from scheduling to execution. The narrative is structured as follows:

Timestamp	Section	Content
0:00 - 0:16	Introduction	Port logistics overview and system objectives
0:16 - 0:46	Algorithm Selection	Five algorithms: Auto, Optimal, Heuristic, Multi-Crane, Genetic
0:46 - 1:04	Rebalancing Demo	MSC Oscar vessel rebalancing from Dock C to Dock A
1:04 - 1:36	Schedule Results	Detailed operation assignments with resources and timing
1:36 - 1:51	Timeline View	Visual 24-hour overview of port operations

Timestamp	Section	Content
1:51 - 2:08	Operation Plans	Search and retrieval by vessel, date, or date range
2:08 - 2:28	Execution Tracking	Real-time monitoring with start/pause/delay controls
2:28 - 2:49	Conclusion	System benefits and value proposition

Total Duration: 2 minutes 49 seconds

6.3 Video Availability and Content Summary

The explanatory video provides a concise, professionally narrated demonstration of the complete port logistics system, showcasing the IARTI scheduling module alongside the broader operational workflow.

Video Link: Port Logistics Scheduling System - Complete Demo

6.3.1 Video Content Highlights

The video presents a complete end-to-end demonstration of the port logistics system:

1. **System Introduction and Context (0:00-0:16)**
    - Overview of port logistics challenges: multiple vessels requiring efficient loading/unloading
    - System objectives: minimize delays and maximize resource utilization
    - Daily operational workflow context
2. **Five Scheduling Algorithms (0:16-0:46)**
    - **Auto Mode:** Recommended for general use, automatically selects best algorithm
    - **Optimal Algorithm:** Finds absolute best solution for small vessel counts
    - **Heuristic Algorithm:** Designed for speed under complex workloads
    - **Multi-Crane Algorithm:** Optimized for crane resource allocation
    - **Genetic Algorithm:** Evolutionary approach for efficiency under varied conditions
    - Target date selection and algorithm choice workflow
3. **Vessel Visit Rebalancing Feature (0:46-1:04)**
    - Real demonstration with MSC Oscar vessel
    - System recommendation: move vessel from Dock C to Dock A
    - Benefits shown: improved balance score and reduced potential delays
    - Emphasis on smart load balancing as key to port efficiency
4. **Comprehensive Schedule Results (1:04-1:36)**
    - Algorithm used and total delay achieved
    - Detailed operation assignments per vessel
    - Resource allocation: specific dock, crane, staff member, storage area
    - Precise timing: start time, end time, duration
    - Calculated delays (minimization demonstrated)
5. **Timeline View Visualization (1:36-1:51)**

    - Intuitive visual representation of all operations
    - 24-hour overview of entire port activity
    - Quick identification of vessel positions and potential conflicts
    - Operator-friendly interface design

6. **Operation Plans Management (1:51-2:08)**

    - Schedule persistence as operation plans
    - Search and retrieval capabilities
    - Filtering options: vessel visit number, specific dates, date ranges
    - Historical tracking of past and present operations

7. **Real-Time Execution Tracking (2:08-2:28)**

    - Execution tracking module for actual vessel operations
    - Operator controls: start, pause, mark as delayed
    - Accurate tracking of all port activities
    - Quick response capability to operational changes
    - Real-time progress monitoring

8. **System Benefits Summary (2:28-2:49)**

    - Integration of intelligent algorithms, resource optimization, and real-time tracking
    - End-to-end optimization from planning to execution
    - Delay minimization and productivity maximization
    - Value proposition for port operations

6.3.2 Production Details

- **Duration:** 2 minutes 49 seconds
- **Format:** Screen recording with professional English narration
- **Style:** Product demonstration with clear, operator-focused narrative
- **Visual Quality:** 1080p HD with smooth transitions
- **Audio:** Clear narration with background system interactions
- **Platform:** YouTube (accessible with link only)
- **Target Audience:** Port operators, logistics managers, stakeholders
- **Production Date:** January 2026

6.3.3 Scope and Integration

Unlike a technical deep-dive, the video adopts a product demonstration approach:

IARTI Module Coverage:

- All five scheduling algorithms (US 4.3.1 Genetic Algorithm included)
- Automatic algorithm selection (US 4.3.2)
- Vessel visit rebalancing workflow (US 4.3.3)

Broader System Integration:

- Complete workflow from scheduling to execution

- Operations Execution Module (OEM) integration
- Resource management and assignment
- Real-time tracking capabilities

Educational and Business Value:

- Demonstrates practical application of IARTI algorithms in production environment
- Shows seamless integration between planning (IARTI) and execution (OEM)
- Provides stakeholder-friendly demonstration of system capabilities
- Illustrates real-world use cases (MSC Oscar rebalancing example)
- Emphasizes operational efficiency and delay reduction metrics

AI Usage Note: The video script was structured with AI assistance to ensure clear narrative flow and comprehensive feature coverage within a concise timeframe. The demonstrations, system interactions, and technical content represent original work by João Sousa (1230655). No AI-generated narration was used; voice-over is human-recorded. (See Appendix A for complete AI usage disclosure.)

---

7. Frontend Integration: Algorithm Comparison Feature

7.1 Overview

A key feature developed in Sprint C is the ability to compare results from multiple scheduling algorithms side-by-side. This helps operators make informed decisions about which solution to apply.

7.2 Implementation

7.2.1 Comparison Modal

Users can select any 2 of the 3 available algorithms to compare:

```
<div class="algorithm-selection">
  <label>Select algorithms to compare:</label>
  <div class="checkbox-group">
    <label>
      <input type="checkbox" [(ngModel)]="compareOptimal"
             [disabled]="!compareOptimal && selectedCount >= 2">
      Optimal (Exhaustive Search)
    </label>
    <label>
      <input type="checkbox" [(ngModel)]="compareHeuristic"
             [disabled]="!compareHeuristic && selectedCount >= 2">
      Heuristic (Greedy Urgency)
    </label>
    <label>
      <input type="checkbox" [(ngModel)]="compareGenetic"
             [disabled]="!compareGenetic && selectedCount >= 2">
      Genetic Algorithm
    </label>
  </div>
</div>
```

7.2.2 Parallel Execution

Selected algorithms are executed in parallel for efficiency:

```
async executeComparison(): Promise<void> {
  const promises: Promise<ScheduleResult>[] = [];

  if (this.compareOptimal) {
    promises.push(this.schedulingService.computeOptimal(this.vessels));
  }
  if (this.compareHeuristic) {
    promises.push(this.schedulingService.computeHeuristic(this.vessels));
  }
  if (this.compareGenetic) {
    promises.push(this.schedulingService.computeGenetic(this.vessels,
this.gaParams));
  }

  const results = await Promise.all(promises);
  this.displayComparisonResults(results);
}
```

7.2.3 Results Display

The comparison modal shows:

- Algorithm name and configuration
- Total delay achieved
- Computation time
- Schedule details (expandable)
- Visual timeline comparison

7.3 User Workflow

1. Click "Compare Algorithms" button
2. Select 2 algorithms from the 3 available options
3. Click "Run Comparison"
4. Review results side-by-side
5. Optionally apply the preferred schedule

---

8. Conclusions

8.1 Sprint C Achievements

Sprint C successfully delivered all planned user stories:

US	Status	Description
4.3.1	☑ Complete	Genetic algorithm implementation with configurable parameters
4.3.2	☑ Complete	Automatic algorithm selection with transparent decision logic
4.3.3	☑ Complete	Vessel visit rebalancing feature with mandatory workflow integration
4.3.4	☑ Complete	Comprehensive state-of-the-art study on robotics and computer vision
4.3.5	☑ Complete	Explanatory video production

8.2 Technical Contributions

Backend Enhancements:

- New genetic algorithm module with multi-dock support
- Automatic algorithm selection engine
- Extended API endpoints for comparison and rebalancing

Frontend Enhancements:

- Algorithm comparison modal with parallel execution
- Rebalancing workflow with visual feedback
- Unified styling across scheduling components

8.3 Integration with Previous Sprints

Sprint C builds upon the foundation of previous sprints:

Sprint A Foundation	Sprint C Enhancement
Optimal algorithm (US 3.4.2)	Integrated into auto-selection and comparison
Heuristic algorithm (US 3.4.4)	Integrated into auto-selection and comparison
Multi-crane mode (US 3.4.5)	Optional integration with genetic algorithm

8.4 Lessons Learned

1. **Algorithm Performance:** Genetic algorithms provide excellent trade-off between quality and speed for medium-large instances
2. **User Experience:** Mandatory rebalancing check ensures operators follow best practices but requires clear UI feedback
3. **Modular Architecture:** Independent dock processing in GA enables scalability and potential parallelization

---

9. References

9.1 Academic References

1. Holland, J. H. (1992). Adaptation in Natural and Artificial Systems. MIT Press.
2. Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.
3. Bierwirth, C., & Meisel, F. (2015). A survey of berth allocation and quay crane scheduling problems. European Journal of Operational Research, 202(3), 615-627.
4. Kim, K. H., & Park, Y. M. (2004). A crane scheduling method for port container terminals. European Journal of Operational Research, 156(3), 752-768.

9.2 Technical Documentation

5. SWI-Prolog Reference Manual. (2024). https://www.swi-prolog.org/pldoc/
6. Angular Documentation. (2024). https://angular.dev/

9.3 Project Documentation

7. Team 3DB-01. (2025). IARTI Technical Report - Sprint A. ISEP Internal Report.
8. Team 3DB-01. (2025). US 4.3.4 - State of the Art: Robotics and Computer Vision. ISEP Internal Report.

Appendix A: AI Usage Declaration

AI Tool Used: Claude 4.5 Sonnet (Anthropic) and GitHub Copilot

Sections with AI Assistance

Section 2 (US 4.3.1 - Genetic Algorithm):

- AI assistance was used to structure the pseudocode representations and format the algorithm documentation
- Performance benchmarking table formatting
- Technical implementation code snippets organization

Section 3 (US 4.3.2 - Auto Selection):

- AI assistance was used to create the decision flow diagram in ASCII art format
- Selection rules pseudocode formatting and clarity improvements
- JSON response example formatting

Section 5 (US 4.3.4 - State of the Art Study):

- The complete state-of-the-art study is documented in a separate report: US-4.3.4-Robotics-ComputerVision-StateOfArt.md
- That report contains comprehensive AI usage disclosure in its Appendix A, including:
  - Specific prompts used for architectural diagrams and technical enablers categorization
  - Verification process against scientific literature
  - Clear distinction between AI-generated content and original research

Section 6 (US 4.3.5 - Explanatory Video):

- Video script structure and narrative flow were organized with AI assistance
- AI helped optimize the demonstration sequence for clarity and conciseness (2min 49sec duration)
- Specific script elements refined: introduction hook, feature transitions, conclusion message
- All demonstrations, system interactions, screen recordings, and voice narration are 100% original work by João Sousa (1230655)
- No AI-generated voice or synthetic media used; human narration throughout
- Technical content accuracy verified by the team

Report Formatting:

- AI assistance was used to ensure consistent formatting and structure following the Sprint A report template
- Table of contents generation and section numbering
- Markdown formatting for diagrams and code blocks

Verification Statement

All technical implementations, algorithms, code, and system architecture decisions were made independently by the respective team members. AI tools were used exclusively for:

- Documentation clarity and organization
- Diagram formatting and visualization
- Consistent markdown structure
- Literature research assistance (4.3.4)

No AI-generated code was directly integrated into the production system without thorough review and validation by team members.

Report Length: Approximately 20-22 pages (including diagrams, code samples, and appendix)

This report represents original work by Team 3DB-01 for the IARTI course unit. All algorithms, analyses, and implementations are the result of independent development by the team members.